

Lecture 11

Transactions



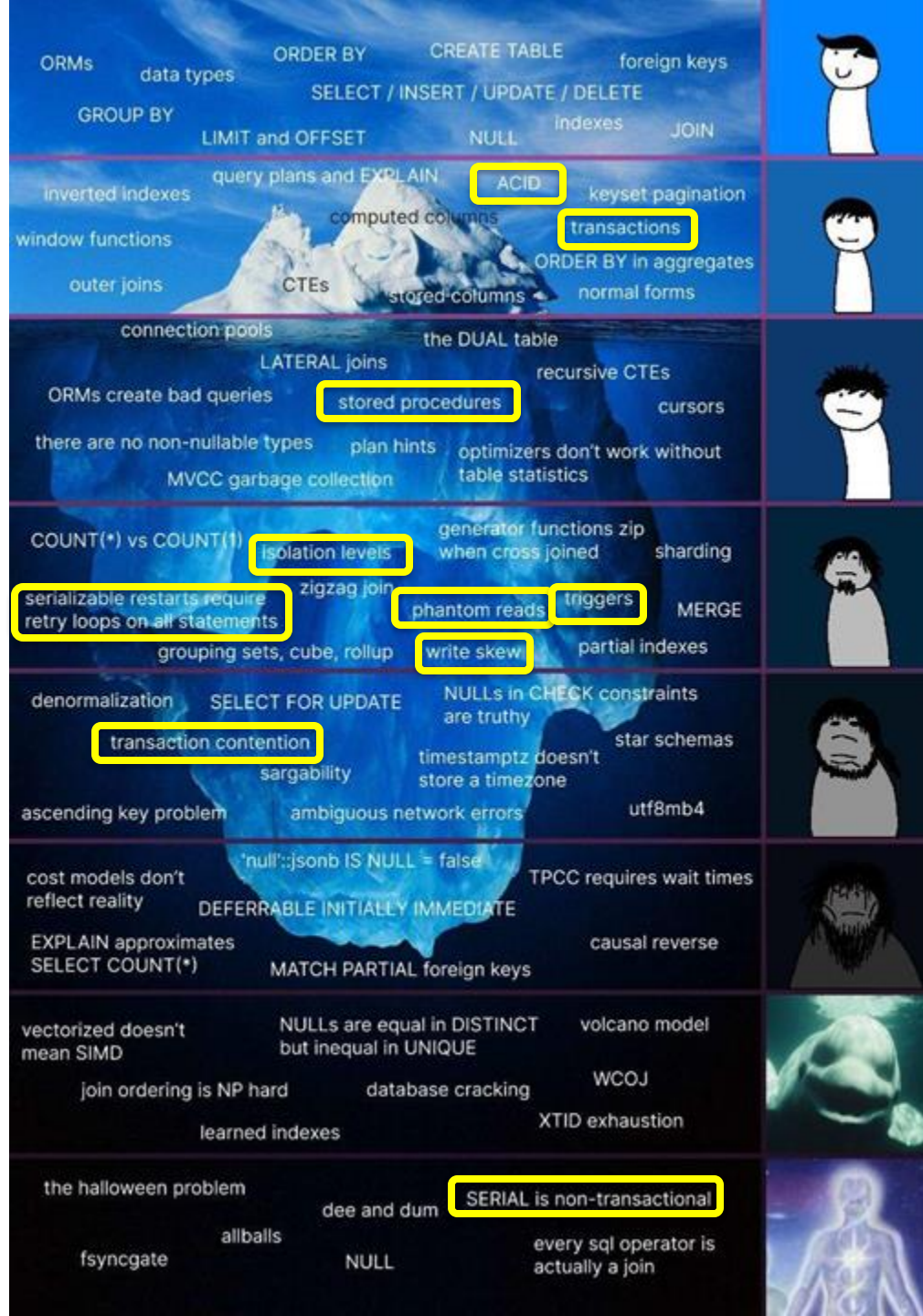
“The Moneylender and His Wife,” Quentin Matsys, 1514

March 17, 2026

Project meetings:
to be scheduled soon!

Where Are We?

- So far:
 - Studied relational model & SQL
 - Learned basic architecture of a database system
 - Studied different operator implementations
 - Looked at several data layouts
 - Saw how query optimizer works with statistics to select plans and operators
- What next:
 - *Concurrency Control and Recovery*: How to ensure correctness in the presence of modifications and failures to the database
 - Distributed and parallel query processing
 - New topics, including AI+data



Next 4 lectures

Transactions

- Group related sequence of actions so they are “all or nothing”
 - If the system crashes, partial effects are not seen
 - Other transactions do not see partial effects
- A set of implementation techniques that provides this abstraction with good performance

ACID Properties of Transactions

- **A** tomicity – many actions look like one; “all or nothing”
- **C** onsistency – database preserves invariants
- **I** solation – concurrent actions don’t see each other’s results
- **D** urability – completed actions in effect after crash (“recoverable”)

Concurrent Programming Is Hard

~~A = 0~~ 1 1

- Example:

T1

t = A

t = t + 1

A = t



T2

t = A

t = t + 1

A = t



- Looks correct!
- But maybe not if other updates to A are interleaved!
- Suppose T1 increment runs just before T2 increment
 - T1 increment will be lost
- Conventional approach: programmer adds locks
 - But must reason about other concurrent programs

Transactions Dramatically Simplify Concurrent Programming

- Guarantees that concurrent actions are *serially equivalent*
 - I.e., appear to have run one after the other
- Programmer does not have to think about what is running at the same time!
- **One of the big ideas in computer science**

SQL Syntax

- **BEGIN TRANSACTION**
 - Followed by SQL operations that modify database
- **COMMIT**: make the effects of the transaction durable
 - After COMMIT returns database guarantees results present even after crash
 - And results are visible to other transactions
- **ABORT**: undo all effects of the transaction

This Lecture: Atomicity

- **Atomicity** – many actions look like one; “all or nothing”
- In reality, actions take time!
 - To get atomicity, we must prevent multiple actions from interfering with each other
 - I.e., they are **I**solated
- Will return to **D**urability in 2 lectures
 - E.g., how to *recover* a database after a crash and bring it to a state where no partial transactions are present

Consistency

- Preservation of invariants
- Usually expressed in terms of constraints
 - E.g., primary keys / foreign keys
 - Triggers
- Example: no employee makes more than their manager
- Requires ugly non-SQL syntax (e.g. PL/pgSQL)
- Often done in the application

Postgres PL/pgSQL Trigger Example

```
CREATE FUNCTION sal_check() RETURNS trigger AS $sal_check$
```

```
DECLARE
```

```
    mgr_sal integer;
```

```
BEGIN
```

```
    IF NEW.salary IS NOT NULL THEN
```

```
        SELECT INTO mgr_sal salary
```

```
        FROM emp
```

```
        JOIN manages
```

```
        ON NEW.eid = manages.eid
```

```
        AND emp.eid = manages.eid
```

```
        LIMIT 1;
```

```
        IF (mgr_sal < NEW.salary) THEN
```

```
            RAISE EXCEPTION 'employee cannot make more than manager';
```

```
        END IF;
```

```
    END IF;
```

```
    RETURN NEW;
```

```
END;
```

```
$sal_check$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER eid_sal BEFORE INSERT OR UPDATE ON emp
```

```
FOR EACH ROW EXECUTE FUNCTION sal_check();
```

NEW is the record being added

mgr_sal is a local variable

Query finds the salary of one manager

Check salary (if no manager, mgr_sal is NULL)

Declare that we want to call sal_check every time a record changes or is added to emp

How Can We Isolate Actions?

- Serialize execution: one transaction at a time
- Problems with this?
 - No ability to use multiple processors
 - Long running transactions *starve* others
- Goal: allow *concurrent* execution while preserving *serial equivalence*
- *Concurrency control* algorithms do this

Serializability

- An ordering of actions in concurrent transactions that is serially equivalent

T1

RA

WA

RB

WB

T2

RA

WA

RB

WB

RA: Read A

WA: Write A, may depend on anything read previously

A/B are “objects” – e.g., records, disk pages, etc

Assume arbitrary application logic between reads and writes

Serially equivalent to T1 then T2

Serializability

- An ordering of actions in concurrent transactions that is serially equivalent

T1
RA

T2

RA
WA

RA: Read A

WA: Write A, may depend on anything read previously

WA

RB

WB

A/B are “objects” – e.g., records, disk pages, etc

Assume arbitrary application logic between reads and writes

RB
WB

Not serially equivalent – T2’s write to A is lost, which can’t occur in a serial schedule

In T1->T2, T2 should see T1’s write to A

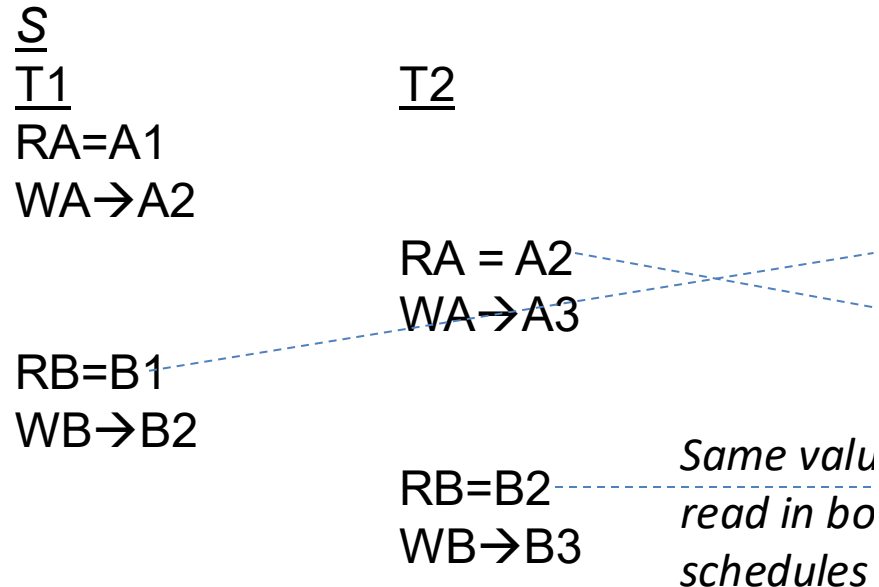
In T2->T1, T1 should see T2’s write to A

View Serializability

A particular ordering of instructions in a schedule S is *view equivalent* to a serial ordering S' iff:

- Every value read in S is the same value that was read by the same read in S' .
- The final write of every object is done by the same transaction T in S and S'
- Less formally, all transactions in S “view” the same values they view in S' , and the final state after the transactions run is the same.

View Serializability Example



Every value read in S is the same value that was read by the same read in S' .

The final write of every object is done by the same transaction T in S and S'

<https://clicker.mit.edu/6.5830/>

Is the following schedule
view serializable?

T1	T2
RA=A1	
	RA=A1
	WA->A2
	WB->B2
WB->B3	

Is it view equivalent to
T1->T2?

What about T2->T1?

Not T1->T2 because at
left, T1 writes final
value of B

Not T2->T1 because at
left, T1's RA gets A1
rather than A2

A) Yes

B) No

C) I don't know

A particular ordering of instructions in a schedule S is *view equivalent* to a serial ordering S' iff:

- Every value read in S is the same value that was read by the same read in S' .
- The final write of every object is done by the same transaction T in S and S'

View Serializability Limitations

- Must test against each possible serial schedule to determine serial equivalence
 - NP-Hard! *(For N concurrent transactions, there are 2^N possible serial schedules)*
- No protocol to ensure view serializability as transactions run
- *Conflict serializability* addresses both points

Conflicting Operations

- Two operations are said to conflict if:
 - Both operations are on the same object
 - At least one operation is a write
- E.g.,
 - $T1_{WA}$ conflicts with $T2_{RA}$, but
 - $T1_{RA}$ does not conflict with $T2_{RA}$

T2 \ T1	R	W
R	✓	✗
W	✗	✗

Conflict Serializability

A schedule is *conflict serializable* if it is possible to swap non-conflicting operations to derive a serial schedule.

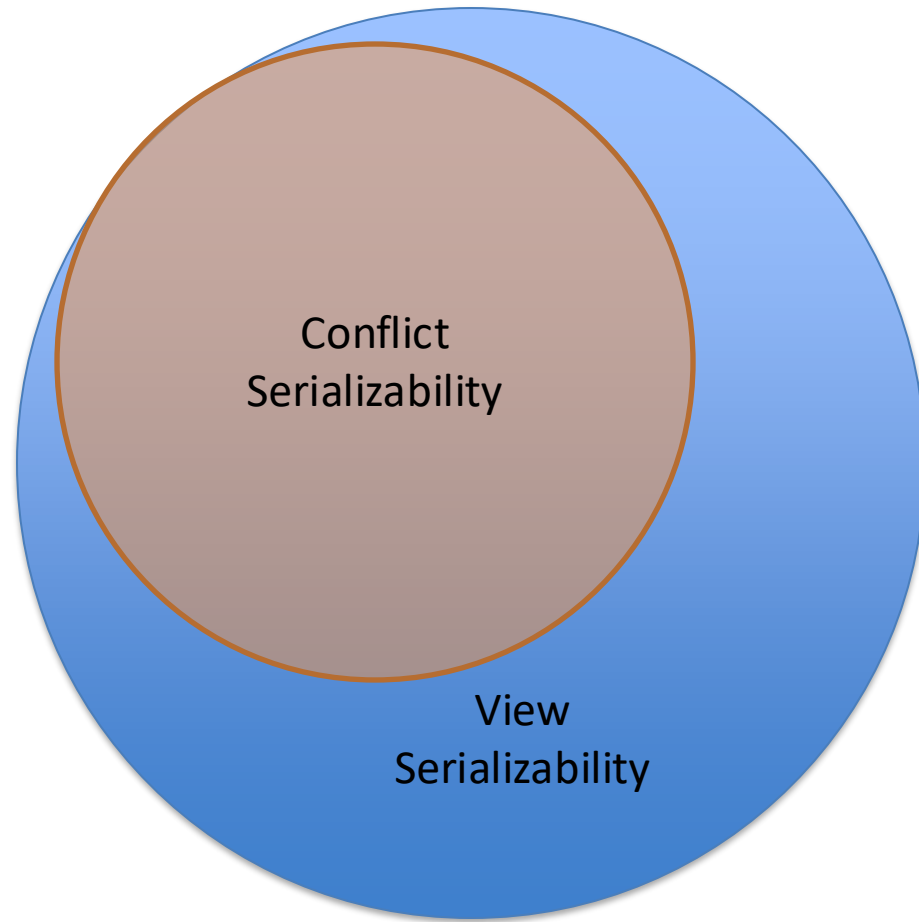
Equivalently

For all pairs of conflicting operations {O1 in T1, O2 in T2} either

- O1 always precedes O2, or
- O2 always precedes O1.

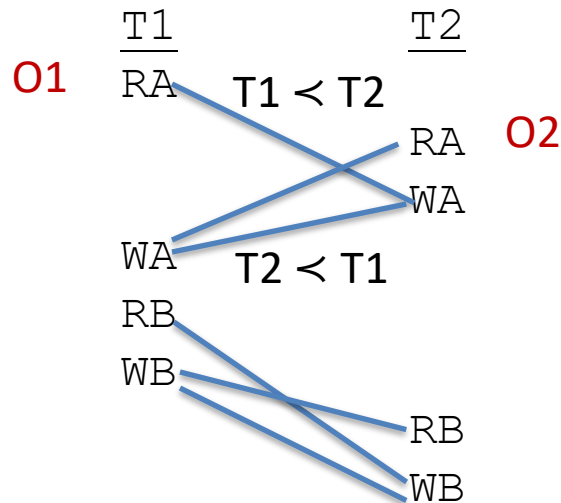
$T1 \prec T2$: “T1 precedes T2”

Testing for Serializability

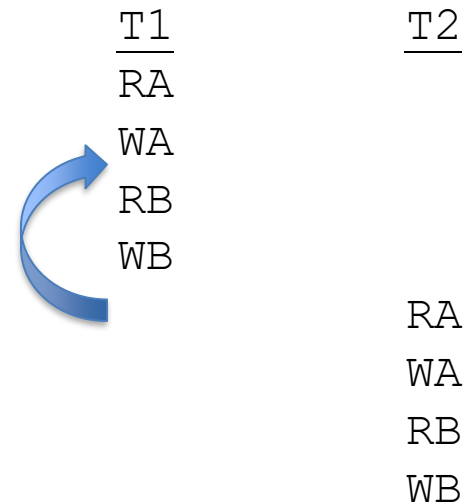


*Any schedule that is **conflict serializable** is **view serializable**, but not vice-versa*

Example



Not conflict serializable!



In conflict serializable schedule,
can reorder non-conflicting ops to
get serial schedule

For all pairs of conflicting operations {O1 in T1, O2 in T2} either
O1 always precedes O2, or
O2 always precedes O1.

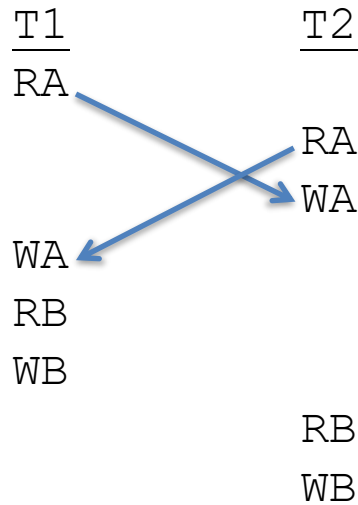
Precedence Graph

Given transactions T_i and T_j ,
Create an edge from $T_i \rightarrow T_j$ if:

- T_i reads/writes some A before T_j writes A
 $RA_{T_i} < WA_{T_j}$ or $WA_{T_i} < WA_{T_j}$
or
- T_i writes some A before T_j reads A
 $WA_{T_i} < RA_{T_j}$

If there are cycles in this graph, schedule is not conflict serializable

Non-Serializable Example



Precedence Graph



Cycle!

Create an edge from $T_i \rightarrow T_j$ if:

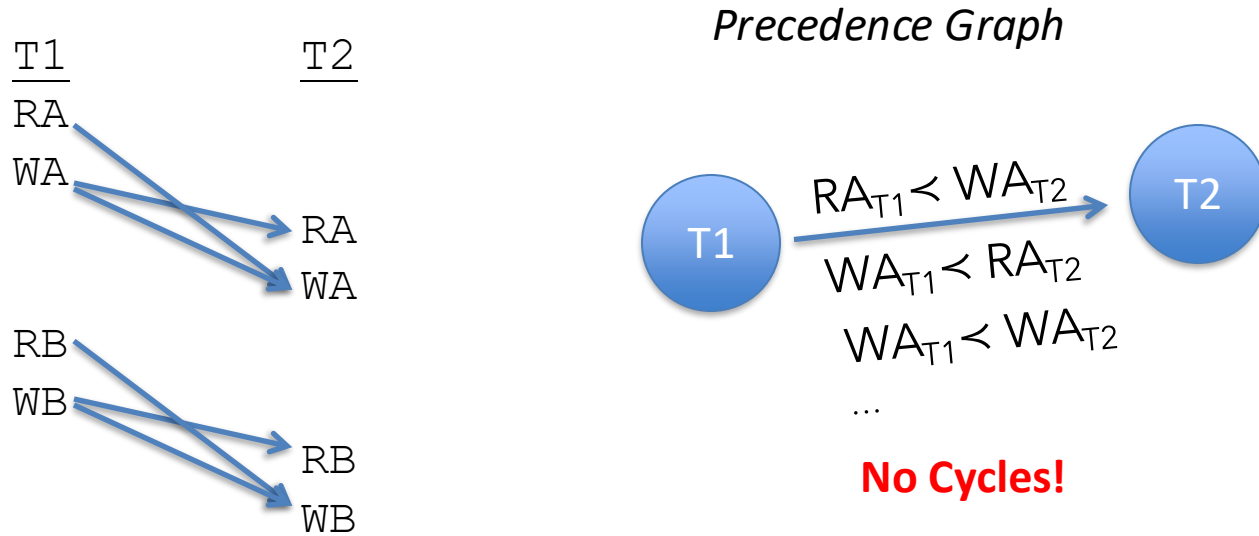
T_i reads/writes some A before T_j writes A, or

$$RA_{T_i} < WA_{T_j} \text{ or } WA_{T_i} < WA_{T_j}$$

T_i writes some A before T_j reads A

$$WA_{T_i} < RA_{T_j}$$

Serializable Example



Create an edge from $T_i \rightarrow T_j$ if:

T_i reads/writes some A before T_j writes A, or

$$RA_{T_i} < WA_{T_j} \text{ or } WA_{T_i} < WA_{T_j}$$

T_i writes some A before T_j reads A

$$WA_{T_i} < RA_{T_j}$$

Recap: 3 Ways to Test for Conflict Serializability

1. Check: For all pairs of conflicting operations $\{O1 \text{ in } T1, O2 \text{ in } T2\}$ either
 1. $O1$ always precedes $O2$, or
 2. $O2$ always precedes $O1$.
2. Swap non-conflicting operations to get serial schedule
3. Build precedence graph, check for cycles

Clicker:

<https://clicker.mit.edu/6.5830/>

- Is this schedule conflict serializable?

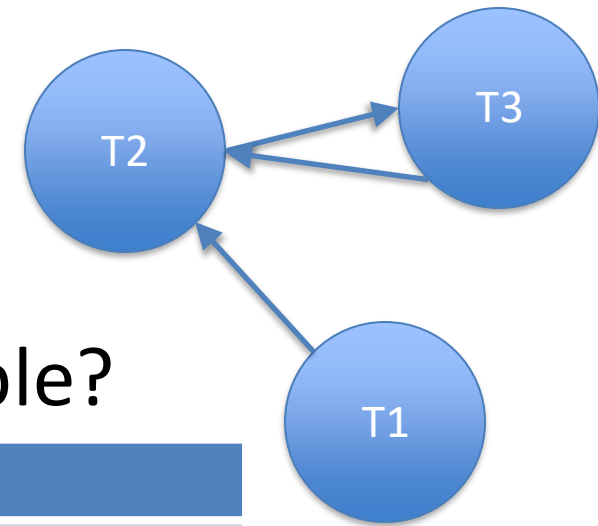
A) Yes

B) No

C) ???

T1	T2	T3
RA		
	RB	
WA		
		RB
	WB	
		WB
	RA	
	WA	
COMMIT	COMMIT	COMMIT

Clicker



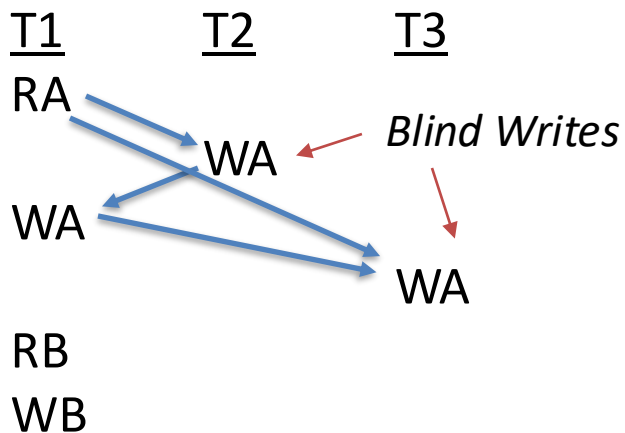
- Is this schedule conflict serializable?

T1	T2	T3
RA		
	RB	
WA		
		RB
	WB	
		WB
	RA	
	WA	
COMMIT	COMMIT	COMMIT

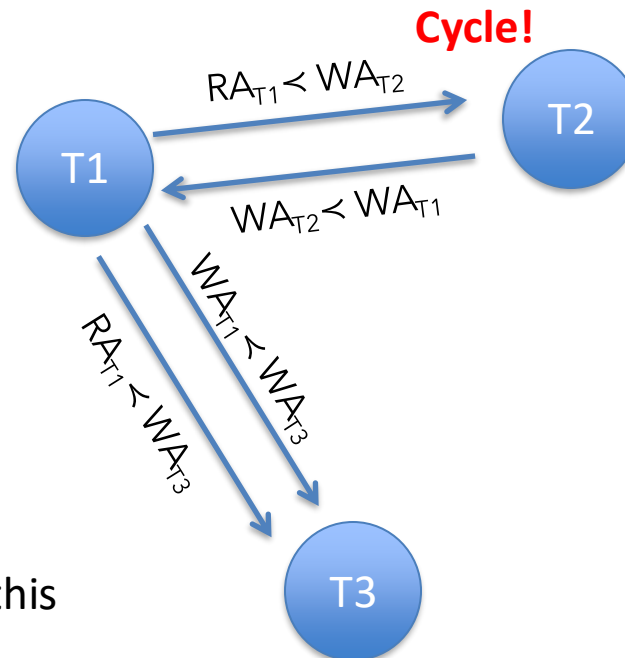
No!

View vs Conflict Serializable

- Testing for view serializability is NP-Hard
 - Have to consider all possible orderings
- Conflict serializability used in practice
 - Not because of NP-Hardness
 - Because we have a way to enforce it as transactions run
- Example of schedule that is view serializable but not conflict serializable:



Equivalent to $T1, T2, T3$
Conflict serializability does not permit this
Only happens with *blind writes*



Implementing Conflict Serializability

- Several different protocols
- Today: Two Phase Locking (2PL)
- Basic idea:
 - Acquire a shared (S) lock before each read of an object
 - Acquire an exclusive (X) lock before each write of an object
- Several transactions can hold an S lock
- Only one transaction can hold an X lock
- If a transaction cannot acquire a lock it waits (“blocks”)

T2 \ T1	R	W
R	✓	✗
W	✗	✗



T2 \ T1	S	X
S	✓	✗
X	✗	✗

Lock Compatibility Table

Conflicting operations (from def. of conflict serializability) are not compatible with each other

When to Release Locks

- After each op completes?
- Or after xaction is done with variable?
- No! Example of problem →
- T2 “sneaks in” and updates A and B before T1 updates B

T1

Xlock A

RA

WA

Rel A

T2

Xlock A

RA

WA

Xlock B

RB

WB

Rel A,B

Xlock B

RB

WB

Rel B

This schedule is not serializable

Solution: Two Phase Locking

- *A transaction cannot release **any** locks until it has acquired **all** of its locks*
- Two-phase locking has a ***growing phase*** and a ***shrinking phase***

Example, Revisited

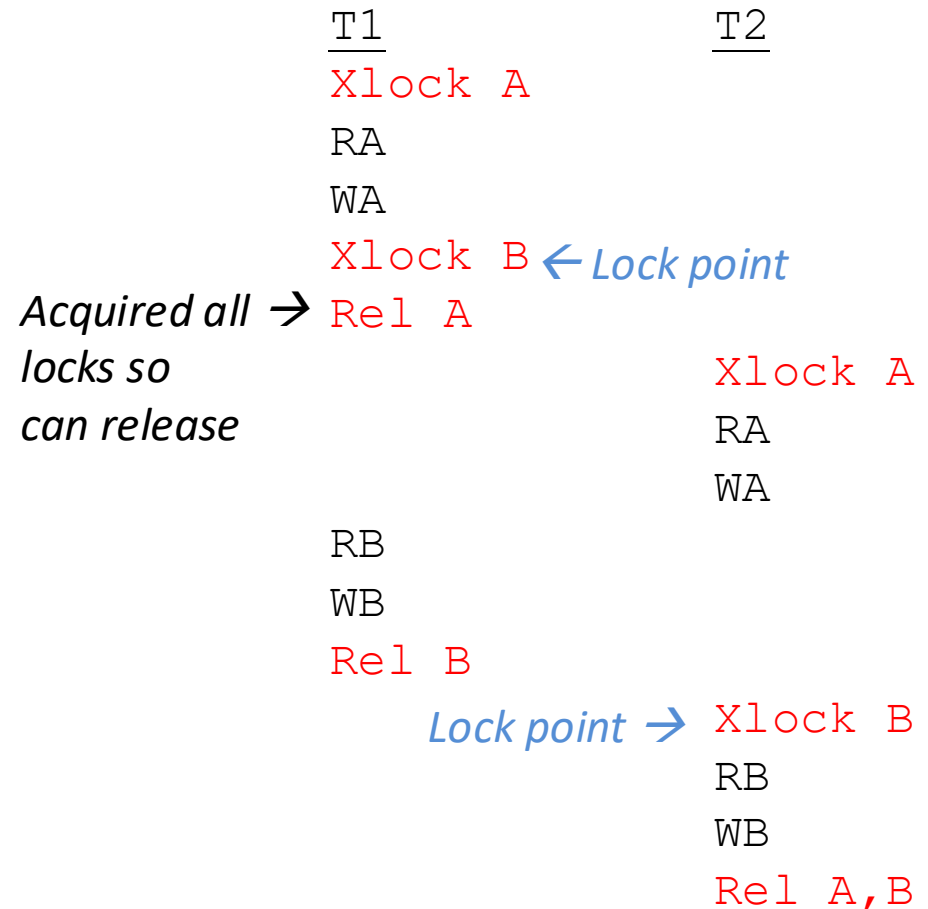
- Rule: A transaction cannot release any locks until it has acquired all of its locks

<u>T1</u>		<u>T2</u>
Xlock A		
RA		
WA		
Not allowed → Rel A		
		Xlock A
		RA
		WA
		Xlock B
		RB
		WB
		Rel A, B
Xlock B		
RB		
WB		
Rel B		

This schedule is not serializable

Example, Revisited

- Rule: A transaction cannot release any locks until it has acquired all of its locks
- Serial schedule defined by *lock points*
 - Where they acquire last lock



*This schedule *is* serializable*

Correctness Intuition

- Once a transaction T reached its lock point...
 - T's place in serial order is set
 - Any transactions that haven't acquired all their locks can't take any conflicting actions until after T releases locks
 - **Ordered later**
 - Any transactions which already have all their locks must have completed their conflicting actions (released their locks) before T can proceed
 - **Ordered earlier**

Two Phase Locking (2PL) Protocol

- Before every read, acquire a shared lock
- Before every write, acquire an exclusive lock (or "upgrade") a shared to an exclusive lock
- Release locks only after last lock has been acquired, and ops on that object are finished

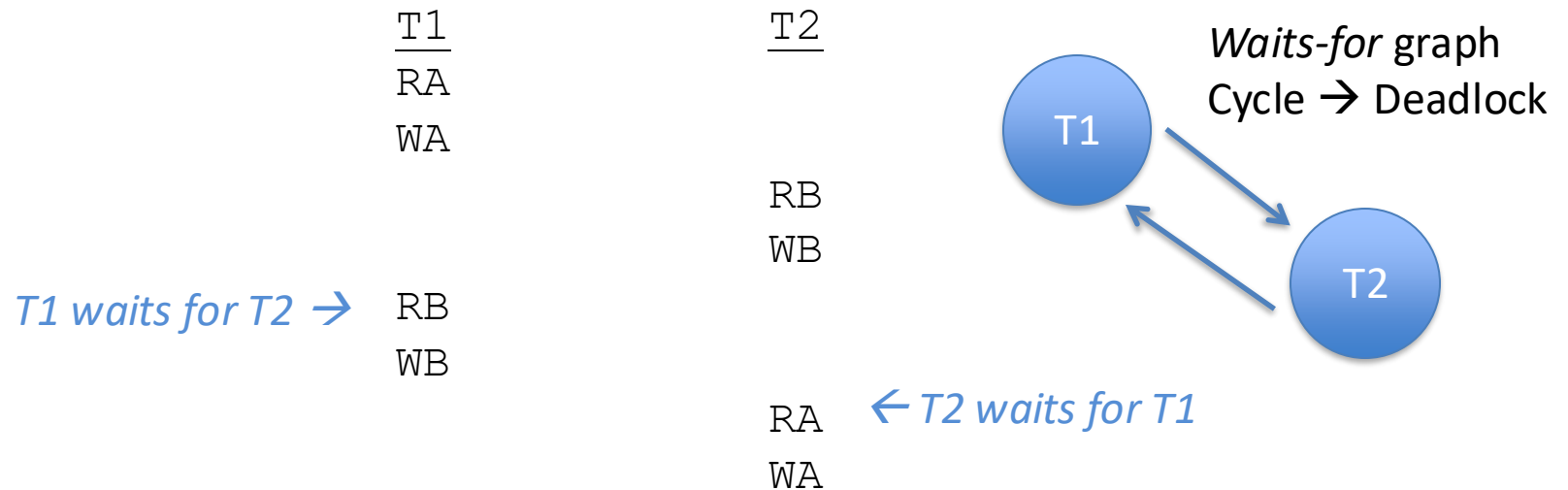
Can you think of any potential problems with 2PL?

Refining 2PL

- Problems:
 - Deadlocks
 - Cascading Aborts
 - How do we know when we are done with all operations on an object?

Deadlocks

- Possible for T_i to hold a lock T_j needs, and vice versa



Complex Deadlocks Are Possible

T1
RA
WA

T2

RB
WB

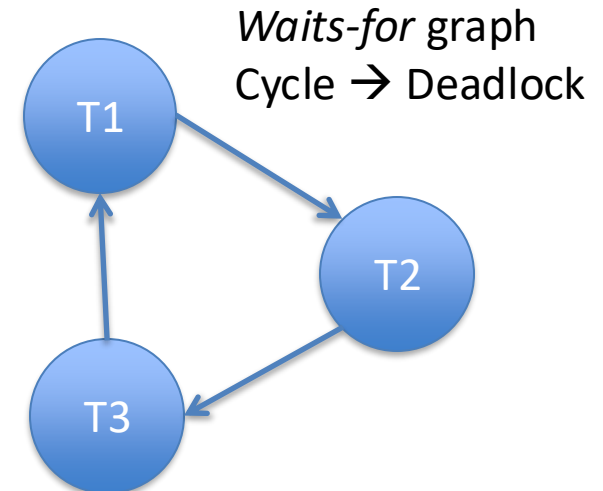
T3

RC

RA ← *T3 waits for T1*
WA

T1 waits for T2 → RB
WB

RC ← *T2 waits for T3*
WC



Resolving Deadlock

- Solution: abort one of the transactions
 - Recall: users can abort too

T1
RA
WA

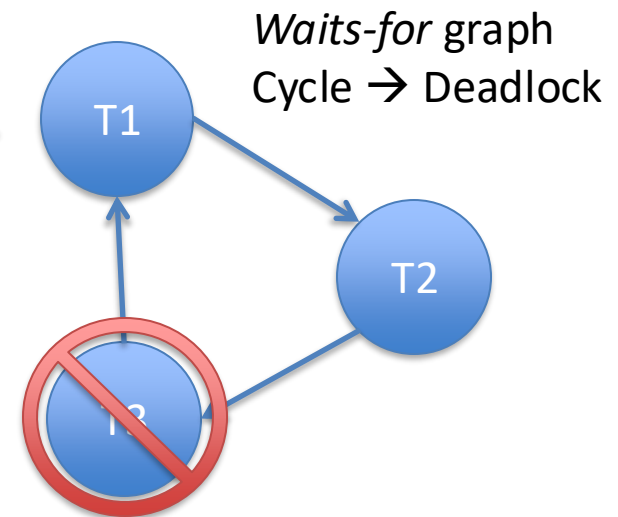
T2

RB
WB

T1 waits for T2 → RB
WB

RC
WC ← ~~*T2 waits for T3*~~

Equivalent to T2 - T1



Cascading Aborts

- Problem: if T1 aborts, and T2 read something T1 wrote, T2 also needs to abort

T1

Xlock A

RA

WA

Xlock B

Rel A

T2

Xlock A

RA

WA

*If T1 aborts here →
T2 also needs to abort,
as it reads T1's write of A*

RB

WB

Rel B

Xlock B

RB

WB

Rel A, B

**Can you think of a 2PL variant which
neither requires deadlock detection nor
has cascading aborts?**

Strict Two-Phase Locking

- Can avoid cascading aborts by holding exclusive locks until end of transaction
- Ensures that transactions never read other transaction's uncommitted data

Strict Two-Phase Locking Protocol

- Before every read, acquire a shared lock
- Before every write, acquire an exclusive lock (or "upgrade") a shared to an exclusive lock
- ~~• Release locks only after last lock has been acquired, and ops on that object are finished~~
- Release *shared* locks only after last lock has been acquired, and ops on that object are finished
- Release *exclusive* locks only after the transaction commits
- Ensures cascadeless-ness

Problem: When is it OK to release?

- How does DBMS know a transaction no longer needs a lock?
- Difficult, since transactions can be issued interactively
- In practice, this means that all locks held until end of transaction
- This is called *rigorous two-phase locking*

Rigorous Two-Phase Locking Protocol

- Before every read, acquire a shared lock
- Before every write, acquire an exclusive lock (or "upgrade") a shared to an exclusive lock
- Release locks only after the transaction commits
- Ensures cascadeless-ness, and
- *Commit order = serialization order*

Next Lectures

- Optimistic concurrency control: Another protocol to achieve conflict serializability
- Nuances that arise with locking granularity